Practical Implementation and Report

1) Introduction

The purpose of this report is to conduct a comprehensive data analysis focused on machine learning techniques, specifically R coding focused on the Wisconsin breast cancer dataset. The whole point of this dataset is to aid in predicting whether a tumour is benign or malignant. Benign tumours will grow but will not spread to other parts of your body where malignant tumours can do so (Cleveland, 2021). This data analysis will focus on pre-processing and the data used, machine learning methods and models as well as showcasing practical content relating to these models. In particular, the aim is to explore the applications of these machine learning models using the likes of decision trees or logistical regression to help see and classify the tumours from our given dataset accurately. Along with this, the goal is to show the effectiveness of machine learning in a wide range of sectors, but in this case during medical applications.

2) Data Used

The dataset provided and used for analysis and the machine learning (ML) models is employed from the University of Wisconsin Hospitals. It consists of a series of different images derived from digitised images of a fine needle aspirate (FNA) of breast masses which were obtained through clinical studies at the hospital (Wolberg et al., 1995). This particular dataset contains both categorical and numerical values throughout. The categorical values include the diagnostic outcomes of ‘benign’ and ‘malignant’ labels as well as indicating whether these tumours are cancerous or not. As for the numerical values these include radius, texture and smoothness amongst several others. These are typically represented as either integers or floating point numbers.

Prior research using this dataset has demonstrated the efficiency of ML models to accurately diagnose tumours based on this. Research using ML classifiers in the medical domain has been prevalent for a while and results from these previous studies demonstrated using classifiers produced good classification accuracies (Abdulkareem & Abdulkareem, 2021).

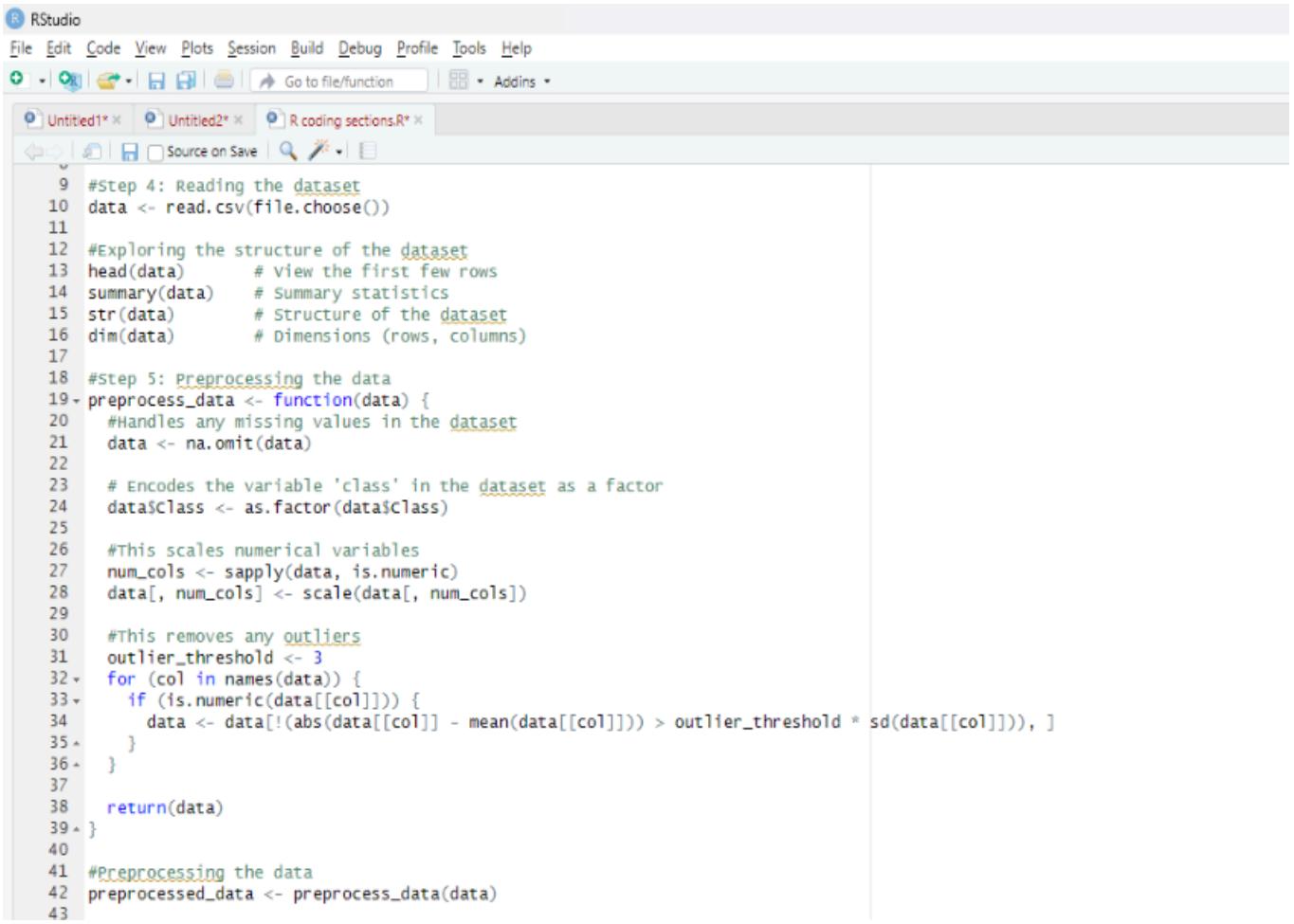
3) Machine Learning Methods Used

For the data analysis several machine learning techniques will be employed. These machine learning techniques include the likes of logistic regression and decision trees. Logistic regression refers to a widely used algorithm for binary classification tasks (Edgar & Manz, 2017) which is present in this dataset. These are useful in medical environments due to their simplicity and interpretability.

Next is decision trees, these are also powerful and easy to understand machine learning models that excel at capturing complex relationships between features and are also particularly useful for classification tasks.

In order to measure the success of these ML methods these following evaluation metrics were used; accuracy, precision, recall and also F1 score. Accuracy refers to the models correctness to classifying tumours. Precision is the correctly predicted case amongst all predicted cases. Recall which is also known as sensitivity is the proportion of all predicted malignant cases amongst actual malignant cases and F1 score is the mean of precision and recall which provides an overall measure of the models performance. These metrics were calculated using a 'confusionMatrix' function. A confusion matrix visualises and summarises the performance of a classification algorithm (Singh et al., 2021).

4) Practical: Pre-Processing Of data



Firstly, the dataset was read in using the read.csv(file.choose)' function in R studio as shown in the image above. This allows a file to be chosen which was the wisconsin.csv dataset file. It loaded the data into the R environment for machine learning analysis.

In the above image I also did preprocessing to handle any missing values and to encode the variable ‘class’ in the dataset as a factor. To handle the missing values a function was used to remove any rows with missing data. Then when encoding the ‘class’ variable as a function it can convert categorical data into a format that can be used by the ML algorithms.

```
66
67  #Check the lengths of the vectors: I did this because it kept on returning errors saying they are not the same length, one was 135 and one 139
68  length(logistic_predictions)
69  length(test_data$Class)
70
71  #Check unique values of class labels in logistic predictions
72  unique(logistic_predictions)
73
74  #Check unique values of class labels in test data
75  unique(test_data$Class)
76
77  #Check for missing values in logistic predictions, I did this because i wanted to know if there was any before.
78  sum(is.na(logistic_predictions))
79
80  #Check for missing values in test data
81  sum(is.na(test_data$Class))
82
83  #Check the length of the original test data, there were 4 missing values here
84  nrow(test_data)
85
86  #Check the length of logistic predictions
87  length(logistic_predictions)
88
89  #Check for missing values in the original test data
90  sum(is.na(test_data))
91
92  #Identify numeric columns
93  numeric_cols <- sapply(test_data, is.numeric)
94
95  #Impute missing values with mean for numeric columns
96  test_data_imputed <- test_data
97  test_data_imputed[, numeric_cols] <- lapply(test_data_imputed[, numeric_cols], function(x) {
98    ifelse(is.na(x), mean(x, na.rm = TRUE), x)
99  })
100
143:2  (Top Level)
```

However, when I was attempting to build the ML models it repeatedly returned errors saying the lengths of the data were not the same, so I had to undergo some more preprocessing as shown in this image above. On lines 84 and 87 were where the length of the values in the original data and logistical prediction data were found to be different in lengths. Line 90 showcased that there were 4 missing values. This snippet of code in the image below shows where this was found.

```
R 4.3.3 · ~/
[1] 0
> # Check for missing values in test data
> sum(is.na(test_data$Class))
[1] 0
> # Check the length of the original test data
> nrow(test_data)
[1] 139
> # Check the length of logistic predictions
> length(logistic_predictions)
[1] 135
> # Check for missing values in the original test data
> sum(is.na(test_data))
[1] 4
```

The snippet of code below shows the further preprocessing done in order to rectify the 4 missing values which were earlier shown to be missing above.

```r
#Identify numeric columns
numeric_cols <- sapply(test_data, is.numeric)

#Impute missing values with mean for numeric columns
test_data_imputed <- test_data
test_data_imputed[, numeric_cols] <- lapply(test_data_imputed[, numeric_cols], function(x) {
  ifelse(is.na(x), mean(x, na.rm = TRUE), x)
})

#Now, let's check again for missing values
sum(is.na(test_data_imputed))
```

5) Practical: R Programming Content

```r
#Load necessary libraries
library(caret)
```

```r
#Make predictions using logistic model
logistic_predictions <- predict(logistic_model, test_data_imputed)

#Compute evaluation metrics
logistic_eval_metrics <- confusionMatrix(logistic_predictions, test_data_imputed$Class)
logistic_accuracy <- logistic_eval_metrics$overall["Accuracy"]
logistic_precision <- logistic_eval_metrics$byClass["Pos Pred Value"]
logistic_recall <- logistic_eval_metrics$byClass["Sensitivity"]
logistic_f1 <- logistic_eval_metrics$byClass["F1"]

#Print evaluation metrics
cat("Logistic Regression Model Evaluation Metrics:\n")
cat("Accuracy:", logistic_accuracy, "\n")
cat("Precision:", logistic_precision, "\n")
cat("Recall:", logistic_recall, "\n")
cat("F1-score:", logistic_f1, "\n")

#Generate confusion matrix for logistic regression model
logistic_conf_matrix <- logistic_eval_metrics$table
logistic_conf_matrix

```

In order to train the ML models the package called ‘caret’ was used for this particular dataset. First, the necessary libraries were loaded. Specifically, logistic regression as well as decision tree algorithms were used for the classification tasks. The ‘train()’ function was used in order to build these models. The image above shows the logistic model development. This was done to gain insights into the dataset's characteristics.

```r
#Make predictions using decision tree model
decision_tree_predictions <- predict(decision_tree_model, test_data_imputed)

#Compute evaluation metrics
decision_tree_eval_metrics <- confusionMatrix(decision_tree_predictions, test_data_imputed$Class)
decision_tree_accuracy <- decision_tree_eval_metrics$overall["Accuracy"]
decision_tree_precision <- decision_tree_eval_metrics$byClass["Pos Pred Value"]
decision_tree_recall <- decision_tree_eval_metrics$byClass["Sensitivity"]
decision_tree_f1 <- decision_tree_eval_metrics$byClass["F1"]

#Print evaluation metrics
cat("\nDecision Tree Model Evaluation Metrics:\n")
cat("Accuracy:", decision_tree_accuracy, "\n")
cat("Precision:", decision_tree_precision, "\n")
cat("Recall:", decision_tree_recall, "\n")
cat("F1-score:", decision_tree_f1, "\n")

#Generate confusion matrix for decision tree model
decision_tree_conf_matrix <- decision_tree_eval_metrics$table
decision_tree_conf_matrix
```

This image above shows the decision tree model in development. These are the two machine learning models made for this particular dataset. The ‘caret’ package facilitates the training process as it provides an unified interface for training and evaluation.

After testing these models, their performance was evaluated as mentioned using different evaluation metrics. These metrics induced accuracy, precision, recall and also F1-score. F1 score is the average of precision and recall meaning the best possible F1 score is 1 and the worst 0. A perfect model will yield a result of 1 meaning all predictions are correct (Sharma, 2023). Confusion matrices were generated too in order to visualise the models performance in predicting both benign and malignant cases. Confusion matrices is a performance measurement for ML classification where the output can be two or more classes (Narkhede, 2021). These evaluations help determine which model could potentially be the most useful in a medical application.

The logistic regression model and decision tree model were compared in order to identify which algorithm performed better for this given dataset. This allows for us to see which model performs better in terms of prediction and allows for the assessment of both models’ strengths and weaknesses.

```r
#R Function to generate a bar chart
plot_class_distribution <- function(data) {
  #This loads the necessary library
  library(ggplot2)

  #This generates the bar chart
  ggplot(data, aes(x = Class)) +
    geom_bar(fill = "skyblue", color = "black") +
    labs(title = "Distribution of Classes",
         x = "Class",
         y = "Count")
}

#This calls the function to generate the bar chart
plot_class_distribution(data)
```

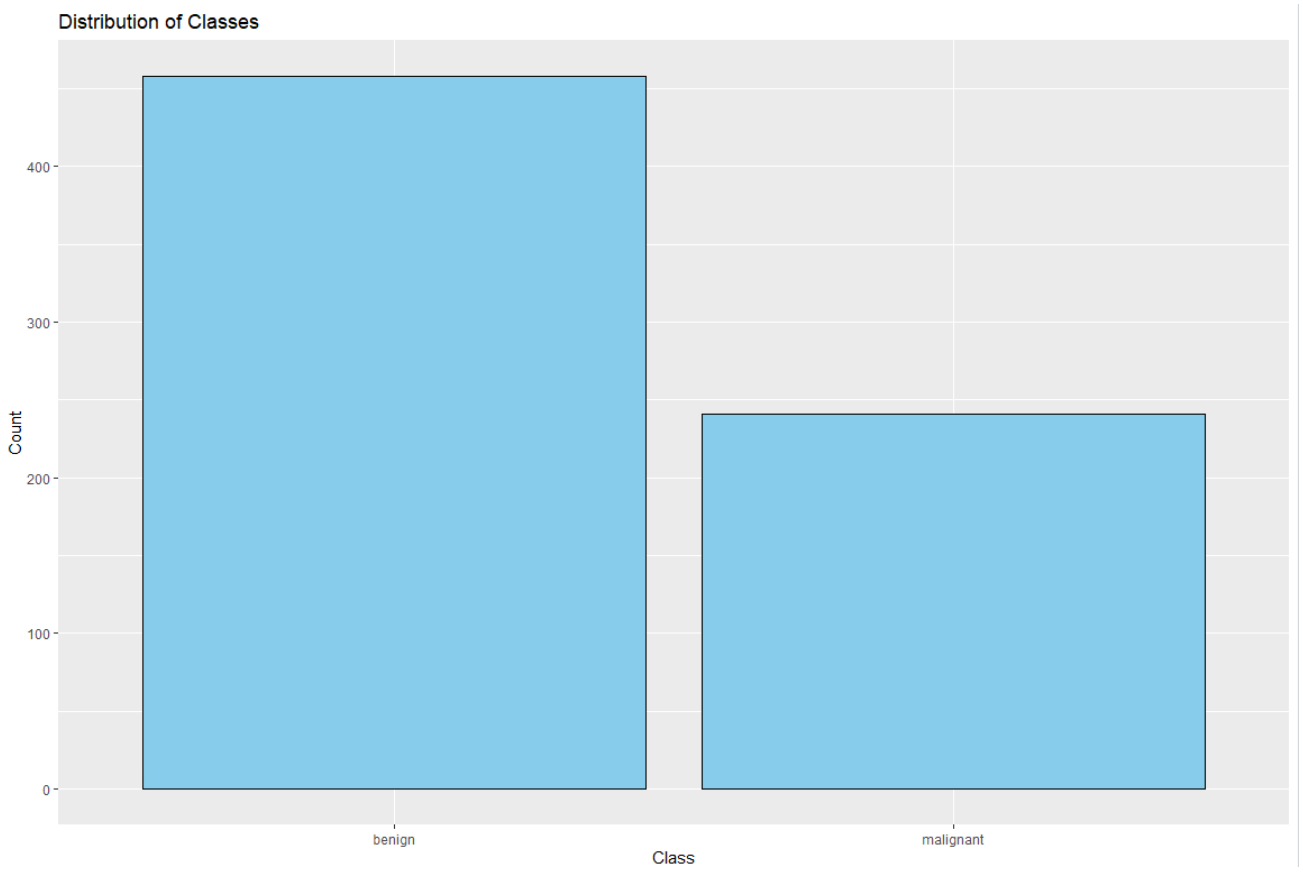
This image above showcases an R function. This function was used in order to develop a bar chart for data analysis and to aid in visualising the distribution of classes in this dataset. This used the ggplot package to create an informative bar chart. By incorporating this R function into the code it aids in providing useful insights into the distribution of classes thus also aiding in model validation.

By doing this it aids the data analysis and also enhances the reproducibility of the data analysis process. This also helps in evaluation.

6) Practical: Display Of Data / Results

The results of the outputs and visualisations obtained from data analysis and the ML process, once again provide valuable insights into the dataset's characteristics and model performance.

Firstly, the R function that was created and mentioned in the previous section was developed in order to form a visualisation of the classes in the dataset. It was a bar chart that was generated. The bar chart helps illustrate a balance or imbalance between the two classes of benign and malignant and helps us understand the datasets class distribution. This graph below is the bar chart formed as a result of the R function that was developed. It clearly shows that there are more benign cases present in the dataset compared to the malignant cases, by roughly double.



Secondly, the evaluation metrics produced from the two ML models were generated to show the models predictions and their 'correctness' level as a result of the confusion matrices.

```
Logistic Regression Model Evaluation Metrics:
> cat("Accuracy:", logistic_accuracy, "\n")
Accuracy: 0.9784173
> cat("Precision:", logistic_precision, "\n")
Precision: 0.9888889
> cat("Recall:", logistic_recall, "\n")
Recall: 0.978022
> cat("F1-score:", logistic_f1, "\n")
F1-score: 0.9834254
> #Generate confusion matrix for logistic regression model
> logistic_conf_matrix <- logistic_eval_metrics$table
> logistic_conf_matrix
           Reference
Prediction  benign malignant
  benign        89         1
  malignant      2        47
```

The above image shows the logistic regression model evaluation metrics. As seen the accuracy is 97.84%, precision 98.84%, recall 97.8% and F1 score 98.34% respectively.

```
Decision Tree Model Evaluation Metrics:
> cat("Accuracy:", decision_tree_accuracy, "\n")
Accuracy: 0.9496403
> cat("Precision:", decision_tree_precision, "\n")
Precision: 0.9883721
> cat("Recall:", decision_tree_recall, "\n")
Recall: 0.9340659
> cat("F1-score:", decision_tree_f1, "\n")
F1-score: 0.960452
> #Generate confusion matrix for decision tree model
> decision_tree_conf_matrix <- decision_tree_eval_metrics$table
> decision_tree_conf_matrix
           Reference
Prediction  benign malignant
  benign        85         1
  malignant      6        47
```

This above image is for the decision tree model evaluation metrics. It shows an accuracy of 94.96%, precision 98.84%, recall 93.41% and F1 score 96.05%. Overall from these two models it seems that the logistic regression model is potentially more reliable than the decision tree model. The bottom of each image also displays the confusion matrices which can help identify which model offers the best balance of correctly predicting the tumour cases.

7) Source Code Listing

```
install.packages("caret")

#Load necessary libraries
library(caret)
library(ggplot2)
################################################################################
#Step 4: Reading the dataset
data <- read.csv(file.choose())

#Exploring the structure of the dataset
head(data)      # View the first few rows
summary(data)   # Summary statistics
str(data)       # Structure of the dataset
dim(data)       # Dimensions (rows, columns)

#R Function to generate a bar chart
plot_class_distribution <- function(data) {
  #This loads the necessary library
  library(ggplot2)

  #This generates the bar chart
```

```
  ggplot(data, aes(x = Class)) +
    geom_bar(fill = "skyblue", color = "black") +
    labs(title = "Distribution of Classes",
         x = "Class",
         y = "Count")
}

#This calls the function to generate the bar chart
plot_class_distribution(data)

################################################################################

#Step 5: Preprocessing the data
preprocess_data <- function(data) {
  #Handles any missing values in the dataset
  data <- na.omit(data)

  # Encodes the variable 'class' in the dataset as a factor
  data$Class <- as.factor(data$Class)

  #This scales numerical variables
  num_cols <- sapply(data, is.numeric)
  data[, num_cols] <- scale(data[, num_cols])

  #This removes any outliers
  outlier_threshold <- 3
  for (col in names(data)) {
    if (is.numeric(data[[col]])) {
      data <- data[!(abs(data[[col]] - mean(data[[col]])) > outlier_threshold * sd(data[[col]])), ]
    }
  }

  return(data)
}

#Preprocessing the data
preprocessed_data <- preprocess_data(data)

#This ceate training and testing datasets
set.seed(123)
train_index <- createDataPartition(data$Class, p = 0.8, list = FALSE)
train_data <- data[train_index, ]
test_data <- data[-train_index, ]

#This trains the models
train_data <- na.omit(train_data)

#Trains the models
logistic_model <- train(Class ~ ., data = train_data, method = "glm", trControl = trainControl(method =
"cv"))
decision_tree_model <- train(Class ~ ., data = train_data, method = "rpart", trControl =
trainControl(method = "cv"))
```

```
################################################################################

#Step 6: The models Evaluation Metrics

#Logistic Regression Model Evaluation
logistic_summary <- summary(logistic_model)
print(logistic_summary)

#Decision Tree Model Evaluation
decision_tree_summary <- summary(decision_tree_model)
print(decision_tree_summary)

#Check the lengths of the vectors: I did this because it kept on returning errors saying they are not the same length, one was 135 and one 139
length(logistic_predictions)
length(test_data$Class)
#Check unique values of class labels in logistic predictions
unique(logistic_predictions)
#Check unique values of class labels in test data
unique(test_data$Class)
#Check for missing values in logistic predictions, I did this because i wanted to know if there was any before.
sum(is.na(logistic_predictions))
#Check for missing values in test data
sum(is.na(test_data$Class))
#Check the length of the original test data, there were 4 missing values here
nrow(test_data)
#Check the length of logistic predictions
length(logistic_predictions)
#Check for missing values in the original test data
sum(is.na(test_data))

#Identify numeric columns
numeric_cols <- sapply(test_data, is.numeric)
#Impute missing values with mean for numeric columns
test_data_imputed <- test_data
test_data_imputed[, numeric_cols] <- lapply(test_data_imputed[, numeric_cols], function(x) {
  ifelse(is.na(x), mean(x, na.rm = TRUE), x)
})

#Checks missing values
sum(is.na(test_data_imputed))


################################################################################

#Make predictions using logistic model
logistic_predictions <- predict(logistic_model, test_data_imputed)

#Computes evaluation metrics
logistic_eval_metrics <- confusionMatrix(logistic_predictions, test_data_imputed$Class)
```

```
logistic_accuracy <- logistic_eval_metrics$overall["Accuracy"]
logistic_precision <- logistic_eval_metrics$byClass["Pos Pred Value"]
logistic_recall <- logistic_eval_metrics$byClass["Sensitivity"]
logistic_f1 <- logistic_eval_metrics$byClass["F1"]

#Print evaluation metrics
cat("Logistic Regression Model Evaluation Metrics:\n")
cat("Accuracy:", logistic_accuracy, "\n")
cat("Precision:", logistic_precision, "\n")
cat("Recall:", logistic_recall, "\n")
cat("F1-score:", logistic_f1, "\n")

#Generate confusion matrix for logistic regression model
logistic_conf_matrix <- logistic_eval_metrics$table
logistic_conf_matrix

################################################################################

#Make predictions using decision tree model
decision_tree_predictions <- predict(decision_tree_model, test_data_imputed)

#Compute evaluation metrics
decision_tree_eval_metrics <- confusionMatrix(decision_tree_predictions, test_data_imputed$Class)
decision_tree_accuracy <- decision_tree_eval_metrics$overall["Accuracy"]
decision_tree_precision <- decision_tree_eval_metrics$byClass["Pos Pred Value"]
decision_tree_recall <- decision_tree_eval_metrics$byClass["Sensitivity"]
decision_tree_f1 <- decision_tree_eval_metrics$byClass["F1"]

#Print evaluation metrics
cat("\nDecision Tree Model Evaluation Metrics:\n")
cat("Accuracy:", decision_tree_accuracy, "\n")
cat("Precision:", decision_tree_precision, "\n")
cat("Recall:", decision_tree_recall, "\n")
cat("F1-score:", decision_tree_f1, "\n")

#Generate confusion matrix for decision tree model
decision_tree_conf_matrix <- decision_tree_eval_metrics$table
decision_tree_conf_matrix
```

8) Conclusions

In conclusion, the analysis of the Wisconsin dataset using the ML models of logistic regression and decision tree models provided promising results. Both models both demonstrated high accuracy, precision, recall and F1 score, suggesting they are both effective in classifying benign and malignant tumours. However, focusing closely on the models it shows the decision tree model isn't as accurate compared to the logistic regression model, however, showed a higher precision percentage indicating lower false positive rates. Both have their own benefits in a clinical setting. The R function provided useful visualisations thus aiding in the visualisation of

results. Overall, both models performed well and are both useful and reliable in real word scenarios.

References

Abdulkareem, S.A. and Abdulkareem, Z.O. (2021) *An Evaluation of the Wisconsin Breast Cancer Dataset using Ensemble Classifiers and RFE Feature Selection Technique Vol 55*. Glasgow: International Journal of Sciences: Basic and Applied Research (IJSBAR).

Cleveland (2021) *Benign tumor: Definition, types, Causes & Management*, *Cleveland Clinic*. Available at: https://my.clevelandclinic.org/health/diseases/22121-benign-tumor (Accessed: 28 February 2024).

Edgar, T. and Manz, D. (2017) *Logistic regression*, *Logistic Regression - an overview | ScienceDirect Topics*. Available at: https://www.sciencedirect.com/topics/computer-science/logistic-regression (Accessed: 29 February 2024).

Narkhede, S. (2021) *Understanding confusion matrix*, *Medium*. Available at: https://towardsdatascience.com/understanding-confusion-matrix-a9ad42dcfd62 (Accessed: 02 March 2024).

Sharma, N. (2023) *Understanding and applying F1 score: A deep dive with hands-on coding*, *Arize AI*. Available at: https://arize.com/blog-course/f1-score/#:~:text=F1%20score%20computes%20the%20average,of%20the%20predictions%20were%20correct. (Accessed: 01 March 2024).

Singh, K. *et al.* (2021) *Machine learning and the internet of medical things in Healthcare* [Preprint]. doi:10.1016/c2019-0-03077-4.

Wolberg, W. *et al.* (1995) *Breast Cancer Wisconsin (Diagnostic)* [Preprint].